Automating layer management



© Copyright 2004-2026, Bootlin
Creative Commons BY-SA 3.0
Corrections, suggestions, contributions and translations are welcome!



Release management

There are multiple tasks that OE/bitbake based projects let you do on your own to ensure build reproducibility:

- Code distribution and project setup
- Release tagging

A separate tool is needed for that, usual solutions are:

- git submodules + setup script. Great example in YOE: https://github.com/YoeDistro/yoe-distro
- repo and `templateconf` or setup script: pretty popular in the past, before dedicated tools emerged
- kas: probably the most popular tool
- bitbake-setup: the newest and official tool



kas

- Specific tool developed by Siemens for OpenEmbedded: https://github.com/siemens/kas
- Will fetch layers and build the image in a single command
- Uses a single JSON or YAML configuration file part of the custom layer
- Can generate and run inside a Docker container
- Can setup `local.conf` and `bblayers.conf`



kas configuration

```
header:
  version: 8
machine: mymachine
distro: mydistro
target:
  - myimage

repos:
  meta-custom:

  bitbake:
    url: "https://git.openembedded.org/bitbake"
    # tag 2.0
    commit: c212b0f3b542efa19f15782421196b7f4b64b0b9
    layers:
      .: excluded

  openembedded-core:
    url: "https://git.openembedded.org/openembedded-core"
    branch: wrynose
    layers:
      meta:
```



kas configuration

```
meta-freescale:
  url: "https://github.com/Freescale/meta-freescale"
  branch: wrynose

meta-openembedded:
 url: https://git.openembedded.org/meta-openembedded
 branch: wrynose
 layers:
   meta-oe:
   meta-python:
   meta-networking:
```

- Then a single command will build all the listed targets for the machine:

```
$ kas build meta-custom/mymachine.yaml
```

- Or, alternatively, invoke `bitbake` commands:

```
$ kas shell /path/to/kas-project.yml -c 'bitbake dosfsutils-native'
```



bitbake-setup

- The official tool, part of bitbake git: https://git.openembedded.org/bitbake/tree/bin/bitbake-setup
- Will fetch layers and setup the build configuration
- Does not try to mask core tools, users still need to launch builds with bitbake
- Uses a JSON configuration file



bitbake-setup configuration

```
{
    "description": "My bitbake setup file.",
    "sources": {
        "bitbake": {
            "git-remote": {
                "uri": "https://git.openembedded.org/bitbake",
                "rev": "2.18"
            }
        },
        "openembedded-core": {
            "git-remote": {
                "uri": "https://git.openembedded.org/openembedded-core",
                "rev": "wrynose"
            }
        },
        "meta-openembedded": {
            "git-remote": {
                "uri": "https://git.openembedded.org/meta-openembedded",
                "rev": "wrynose"
            }
        },
        "meta-freescale": {
            "git-remote": {
                "uri": "https://github.com/Freescale/meta-freescale",
                "rev": "wrynose"
            }
        }
    },
```



bitbake-setup configuration

```
    "bitbake-setup": {
        "configurations": [{
            "name": "myconfig",
            "description": "My own configuration",
            "bb-layers": ["openembedded-core/meta", "meta-openembedded/meta-oe",
                          "meta-openembedded/meta-python", "meta-freescale"],
            "oe-fragments": ["machine/mymachine", "distro/mydistro"]
        }]
    },
    "version": "1.0"
}
```

- Then a single command will fetch all layers and setup your configuration:

```
$ bitbake-setup init mysetup.conf.json
```

- This will also create an environment file that you can source:

```
$ . ./mysetup-myconfig/build/init-build-env
$ bitbake myimage
```